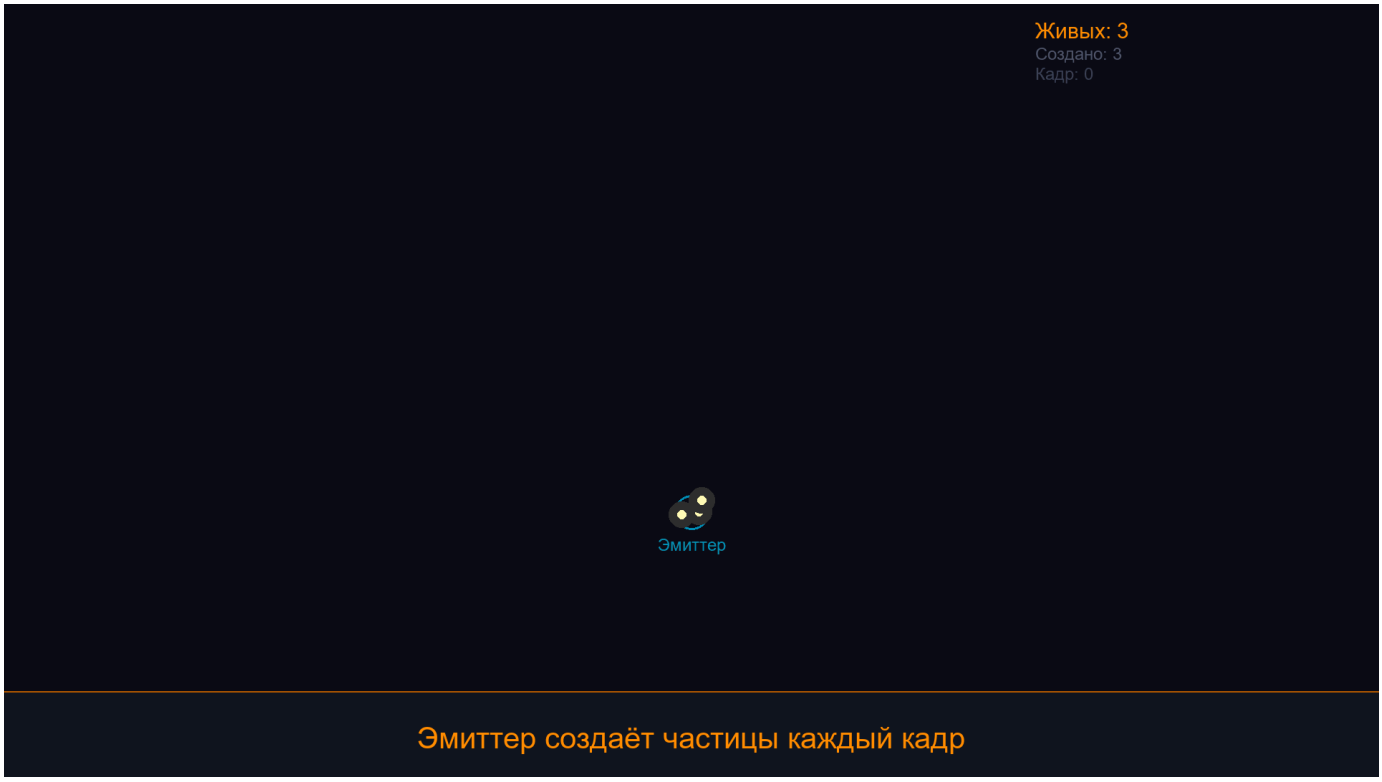
Системы частиц (Particle Systems)

# Системы частиц (Particle Systems)




## Проблема

Герой бьёт врага мечом. Враг теряет здоровье. Но визуально - ничего не происходит. Числа изменились, а на экране всё выглядит так же. Скучно.

Теперь представьте: при ударе из врага летят искры, при взрыве разлетаются обломки, при ходьбе из-под ног поднимается пыль, при смерти врага остаётся облачко дыма. Игра мгновенно ощущается живее. И всё это - одна и та же техника: **системы частиц**.

Частица - это маленький простой объект (точка, квадратик, маленький спрайт, полигон) с позицией, скоростью и временем жизни. По отдельности частица - ничто. Но сотни частиц вместе создают эффекты, которые сложно нарисовать иначе: огонь, дым, дождь, магию.

## Одна частица

Начнём с простого. Что такое частица? Набор данных:

- **Позиция** (x, y) - где находится.
- **Скорость** (vx, vy) - куда и как быстро летит.

- **Время жизни** (lifetime) - сколько секунд осталось до исчезновения.

Каждый кадр: сдвигаем позицию на скорость * deltaTime, уменьшаем время жизни. Когда время жизни <= 0, частица мертва.

Python C#

```python
class Particle:
    # fields: x, y, vx, vy, lifetime, age

    def update(self, delta_time):
        self.x += self.vx * delta_time
        self.y += self.vy * delta_time
        self.age += delta_time

    def is_alive(self):
        return self.age < self.lifetime
```

Вот и всё. Частица - это точка, которая летит и умирает. Пять полей, три строки логики. Вся магия - в том, как мы их порождаем и настраиваем.

## Эмиттер

Одна частица - это ничего. Нам нужен **эмиттер** (emitter) - объект, который порождает множество частиц. Эмиттер знает: где создавать частицы, с какой скоростью, в каком направлении, сколько штук в секунду.

Эмиттер делает три вещи каждый кадр: порождает новые частицы по таймеру, обновляет живые, удаляет мёртвые.

Python C#

```python
# inside ParticleEmitter.update(delta_time):

# 1. spawn by timer
self.spawn_timer += delta_time
interval = 1.0 / self.rate
while self.spawn_timer >= interval:
    self.spawn_timer -= interval
    self._emit()

# 2. update alive
for p in self.particles:
    p.update(delta_time)

```

```
# 3. remove dead
self.particles = [p for p in self.particles if p.is_alive()]
```

```
def _emit(self):
    angle = random.uniform(0, 2 * math.pi)
    spd = random.uniform(0, self.speed)
    vx = math.cos(angle) * spd
    vy = math.sin(angle) * spd
    self.particles.append(Particle(self.x, self.y, vx, vy, self.lifetime))
```

`rate = 50` - 50 частиц в секунду. `speed = 80` - максимальная начальная скорость. `lifetime = 1.0` - каждая частица живёт одну секунду. Направление - случайное (360 градусов). Результат: из точки во все стороны разлетаются оранжевые точки и исчезают через секунду. Уже похоже на огонь или искры.

В `emit()` две случайности: угол ( `cos` / `sin` дают направление) и скорость (разброс по расстоянию). Вместе они превращают одну точку в "облако".

## Одноразовый выброс (Burst)

Непрерывный эмиттер подходит для огня, дыма, дождя. Но взрыв - это не непрерывный процесс. Это одномоментный выброс сотни частиц.

Добавляем метод `burst(count)` - просто вызывает `emit()` в цикле N раз. Создаём эмиттер с `rate = 0` (не порождает сам) и делаем `burst(100)` . Дальше эмиттер только обновляет и удаляет частицы. Через 0.8 секунды все исчезнут.

Если вы реализовали событийную модель, взрыв - идеальный кандидат: опубликовали событие в духе `enemy_died` , отреагировали на него. Враг ничего не знает о частицах, частицы ничего не знают о враге.

## Гравитация и затухание

Частицы, летящие равномерно по прямой, выглядят... скучно. Добавим физику:

**Гравитация** - ускорение вниз. Каждый кадр увеличиваем vy. Частицы начинают лететь вверх, замедляются и падают - как настоящие искры.

**Затухание скорости** (drag) - каждый кадр скорость уменьшается. Частицы замедляются, как будто летят в воздухе.

**Затухание прозрачности** (fade) - чем старше частица, тем прозрачнее. Плавное исчезновение вместо резкого.

В `update` добавляются три формулы:

Python C#

```python
def update(self, delta_time, gravity=0, drag=1.0):
    self.vy += gravity * delta_time
    d = drag ** (delta_time * 60)  # frame-rate independent drag
    self.vx *= d
    self.vy *= d
    # ... same as before: x += vx * dt, y += vy * dt, age += dt

def alpha(self):
    return max(0, 1.0 - self.age / self.lifetime)
```

Зачем `drag ^ (dt * 60)` ? Без этого `drag = 0.98` означает "умножить скорость на 0.98 каждый кадр". При 60 FPS - 60 умножений в секунду: `0.98^60 = 0.30` , за секунду осталось 30% скорости. При 30 FPS - 30 умножений: `0.98^30 = 0.55` , осталось 55%. Частицы на слабом ПК летят дальше.

Формула `0.98^(dt*60)` нормализует к 60 FPS:

- При 60 FPS: `dt=0.016` , `0.98^(0.016×60) = 0.98^1 = 0.98` - минус 2% за кадр, 60 кадров в секунду
- При 30 FPS: `dt=0.033` , `0.98^(0.033×60) = 0.98^2 = 0.96` - минус 4% за кадр, но кадров вдвое меньше

Итого за секунду потеря скорости одинаковая на любом FPS.

С `gravity = 300` и `drag = 0.98` частицы ведут себя как искры от костра: взлетают, замедляются, падают вниз. С `gravity = 0` и `drag = 0.95` - как дым: медленно расползаются и исчезают. Одна и та же частица, разные числа - разные эффекты.

## Рецепты эффектов

Меняя параметры, из одного эмиттера можно получить совсем разные эффекты:

| Эффект | rate | speed | lifetime | gravity | drag | Направление |
|---|---|---|---|---|---|---|
| Огонь | 60 | 40 | 0.8 | -100 | 0.98 | Узкий конус вверх |
| Искры | 0 (burst) | 200 | 0.5 | 300 | 0.97 | 360° |
| Дым | 20 | 20 | 2.0 | -30 | 0.99 | Узкий конус вверх |
| Мед | 0 (burst) | 100 | 0.4 | 400 | 0.95 | 180° (вниз) |

| Эффект | rate | speed | lifetime | gravity | drag | Направление |
|---|---|---|---|---|---|---|
| Дождь | 100 | 300 | 1.5 | 0 | 1.0 | Узкий вниз |
| Пыль из-под ног | 0 (burst 5) | 30 | 0.3 | -20 | 0.9 | В стороны |

Грубо говоря, система частиц - это один код и таблица параметров. Хотите новый эффект? Добавьте строку в таблицу, а не новый класс. Это перекликается с идеей компонентного подхода: данные отдельно, логика отдельно.

Для ограниченного конуса вместо `random.uniform(0, 2 * math.pi)` используйте, например, `random.uniform(-0.3, 0.3) + base_angle`. Это даст разброс в ~35° вокруг нужного направления.

## Спрайтовые частицы

Но все это лишь цветные точки. Работает для искр и простого дыма, но огонь из точек выглядит как конфетти, а не как пламя. Хочется, чтобы каждая частица была маленьким изображением - клубом дыма, язычком пламени, каплей меда.

Идея: вместо `draw.circle` рисуем спрайт. Если вы читали раздел про анимации, то уже знакомы со спрайт-листами. Тот же подход работает и для частиц: все изображения частиц лежат на одном листе, и мы достаём нужный кадр по индексу.

К частице добавляется `frame_index` - индекс кадра на спрайт-листе. При отрисовке вместо `draw.circle` рисуем нужный кадр из листа, умноженный на `alpha()` для затухания. При создании каждой частице назначаем случайный кадр: `frame_index = random(0, 3)`. Четыре варианта дыма - и облако выглядит неоднородным, хотя спрайтов всего четыре.

### Анимированные частицы

А что, если частица не статична, а сама проигрывает анимацию? Клуб дыма расширяется, искра мигает, капля расплёскивается. Каждая частица - маленькая анимация.

Другой вариант - не зацикливать анимацию, а привязать кадр к возрасту частицы. Частица живёт 0.5 секунды, у анимации 4 кадра - значит, кадр переключается каждые 0.125 секунды, и анимация заканчивается ровно когда частица умирает:

```
frame_index = int((age / lifetime) * frame_count)
```

Так дым "от маленького к большому" автоматически совпадает с временем жизни, без подбора frame_duration.

### Текстурный атлас

Когда у вас разные эффекты (дым, искры, огонь, мед), у каждого свой набор спрайтов. Наивный подход: отдельный спрайт-лист на каждый эффект. Но тогда при отрисовке GPU переключается между текстурами - а **каждое переключение текстуры** прерывает **пакетную отрисовку** (batching) и стоит дорого.

Решение: **текстурный атлас** (texture atlas) - одно большое изображение, на котором лежат спрайты всех эффектов сразу. Дым - в левом верхнем углу, искры - правее, огонь - ниже.

```
+--------+--------+--------+
| дым 0  | дым 1  | дым 2  |
+--------+--------+--------+
| огонь0 | огонь1 | огонь2 |
+--------+--------+--------+
| искра0 | искра1 | мед0   |
+--------+--------+--------+
```

Отрисовка всех частиц идёт из одной текстуры. GPU не переключается, `SpriteBatch` / `pygame.blit` работает с одним изображением - всё быстрее.

Это та же идея, что и спрайт-лист из раздела про анимации, только масштабированная на всю игру. Один атлас может содержать и кадры анимации персонажа, и тайлы уровня, и частицы - всё в одном файле. На практике именно так работают большинство 2D игр.

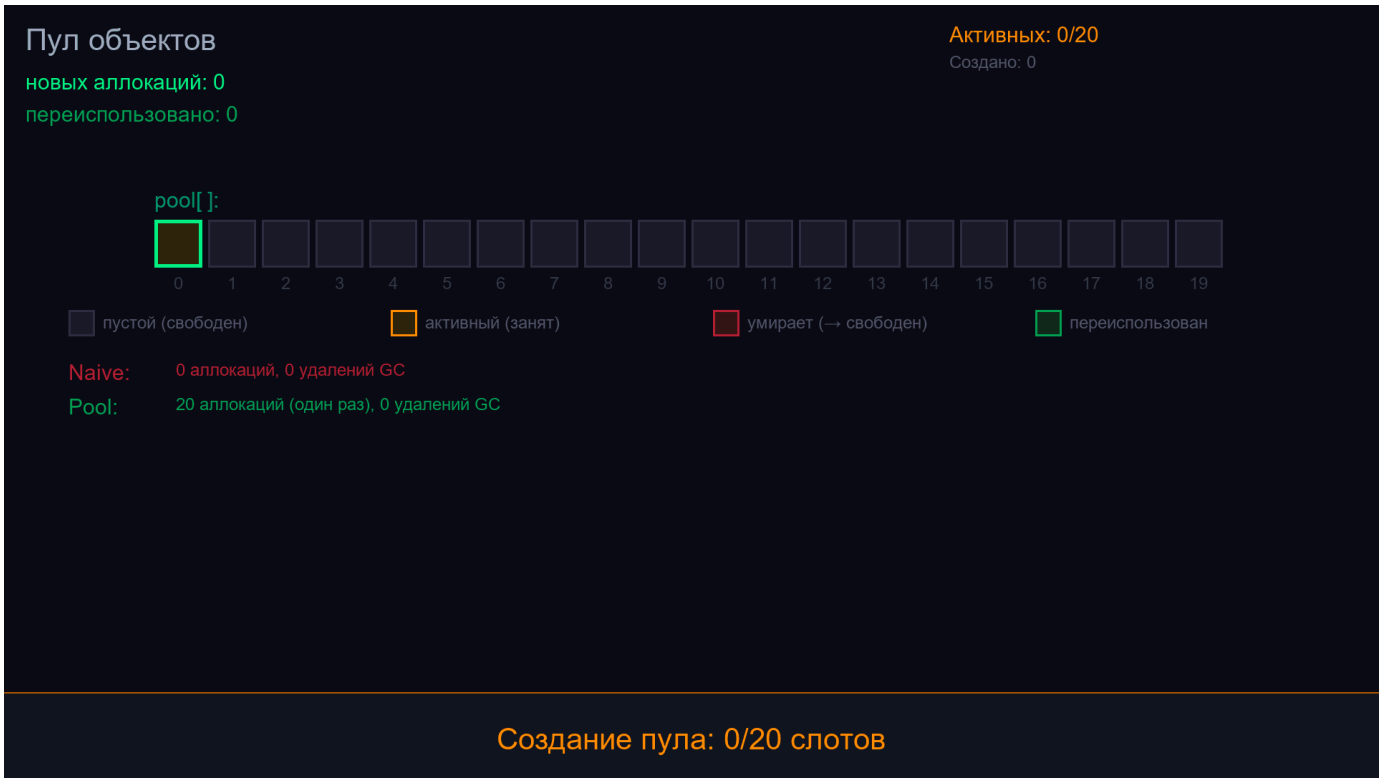
## Производительность: зачем думать о памяти

Ок. У нас работают частицы. 5 эмиттеров по 50 частиц в секунду - 250 новых объектов каждую секунду. Каждый живёт секунду и умирает. Взрывы добавляют ещё сотни.

Что происходит "под капотом"? Каждый `Particle(...)` - это создание нового объекта в памяти. Каждое удаление мёртвой частицы - это работа для **сборщика мусора** (garbage collector, GC). GC периодически останавливает программу, находит мёртвые объекты и освобождает их память. Когда мёртвых объектов сотни в секунду - GC срабатывает чаще, и вы видите микрозависания.

Это не абстрактная теория - это то, от чего ваша игра может начать "подлагивать" каждые пару секунд. Проблема не в алгоритмах, а в том, **как данные живут в памяти**.

### Пул объектов (Object Pool)




Решение: не создавать и не удалять частицы. Вместо этого создаём массив частиц **заранее** - пул. Когда нужна новая частица, берём "мёртвую" из пула и переинициализируем. Когда частица умирает - не удаляем, а помечаем как неактивную. Никаких `new`, никакого GC.

Python C#

```python
class ParticlePool:
    def __init__(self, max_count):
        # all particles pre-allocated, all start "dead"
        self.particles = [Particle(0, 0, 0, 0, 0) for _ in range(max_count

    def spawn(self, x, y, vx, vy, lifetime):
        for p in self.particles:
            if not p.is_alive():
                p.x, p.y = x, y
                p.vx, p.vy = vx, vy
                p.lifetime = lifetime
                p.age = 0.0
                return p
        return None  # pool exhausted
    # update() and draw() iterate only alive particles
```

Создали 500 частиц один раз при старте. `Spawn` не создаёт новый объект, а **переинициализирует** мёртвый. Дальше - ни одного `new`. Сборщик мусора спит. Игра не лагает.

Это называется **пул объектов** (object pool) - один из фундаментальных паттернов оптимизации. Он применяется не только к частицам: пули, снаряды, временные эффекты - всё, что часто создаётся и уничтожается, выигрывает от пула.

Пул - это компромисс: мы тратим больше памяти заранее (500 объектов, даже если активны 50), но получаем стабильную производительность. Если пул закончился, новые частицы просто не появляются - лучше потерять несколько частиц, чем получить микрозависание.

## Данные отдельно, логика отдельно

Обратите внимание: частица - это почти чистые данные. Позиция, скорость, возраст. Логика (обновление, отрисовка) живёт в эмиттере или пуле. Это очень близко к тому, что описано в разделе про ECS: **Component** хранит данные, **System** обрабатывает их.

Более того, все частицы одного типа хранятся в одном массиве, обрабатываются одним циклом. Это называют **data-oriented design** - проектирование, ориентированное на данные. Процессору "нравится" последовательно обрабатывать массив одинаковых структур - он может загрузить их в **кэш** (быструю память рядом с процессором), откуда быстро получить и обработать. Если бы частицы были разбросаны по куче разных объектов в разных местах памяти, каждое обращение к следующей частице было бы "промахом кэша".

Т.е. массив простых структур работает быстрее, чем список сложных объектов - не из-за алгоритма, а из-за того, **как устроена физическая память компьютера**. Это один из тех моментов, где архитектура программы напрямую зависит от архитектуры железа.

Код выше использует `class Particle` - в Python и C# это объекты на куче, не обязательно расположенные в памяти рядом. Это упрощение для наглядности. В настоящих системах частиц данные хранят в плоских массивах ( `struct` в C#, `numpy` -массивы в Python) для настоящей кэш-локальности.

## Итого

Система частиц - это:

1. **Частица** - позиция, скорость, время жизни. Ничего больше.
2. **Эмиттер** - порождает частицы (непрерывно или burst).
3. **Параметры** - гравитация, затухание, направление. Один код, разные эффекты.
4. **Спрайты и анимации** - от точек к изображениям. Текстурный атлас для батчинга.
5. **Пул объектов** - создаём заранее, переиспользуем. Нет `new` - нет лагов от GC.
6. **Data-oriented design** - массив простых данных быстрее, чем куча сложных объектов.

| Что | Где почитать подробнее |
|---|---|
| deltaTime для движения частиц | Основной цикл |
| Событийная модель для запуска эффектов | Событийная модель |
| ECS и data-oriented design | ECS и EC |
| Архитектура: данные vs логика | Архитектура |
| Анимация спрайтовых частиц | Анимации |

Powered by Wiki.js